PostgreSQL Replication Setup

Target: Set up real-time logical replication between PostgreSQL/EDB source and destination with different versions as well as same version.

Prerequisites

- Java 11 or above
- Replication Binaries
- PostgreSQL or EDB installed at source and destination
- JDBC connectors which will comes pre-build with Binaries
- JAR tools: `tabletorepl.jar` and `serverconfig.jar`

Directory Structure

```
/configs
   ├── publication.config
   ├── subscription.config
   ├── tablelist.config
/jars
   ├── tabletorepl.jar
   └── serverconfig.jar
```

Step 1: Setup Replication & Related Services

Ensure these services are configured and started:

1. **Zookeeper**:

   ```
   systemctl start zookeeper
   ```

2. **Kafka Broker**:

   ```
   systemctl start kafka
   ```

3. **Replication Connectors**:

   - Edit `connect-distributed.properties` and set:

   - ```
     key.converter=io.confluent.connect.avro.AvroConverter
     value.converter=io.confluent.connect.avro.AvroConverter
     ```
     ```

```
key.converter.schema.registry.url=http://<Server-ip>:8081
value.converter.schema.registry.url=http:// <Server-ip>:8081
```

- Then start it:

- `systemctl start connect-distributed`

4. **Schema Registry**:

   Edit `schema-registry.properties` or your environment configuration and ensure:

   - `kafkastore.bootstrap.servers=PLAINTEXT://<server-ip>:9092`
   - `listeners=http://0.0.0.0:8081`

   Start the Service:

```
systemctl start schema-registry
```

Step 2: Configure Source Database (Publication)

Edit `publication.config` with the following keys:

```
"name": "",                          // Unique identifier
"tasks.max": "1",                   // Parallelism level
"database.type": "edb",             // edb or postgresql
"database.hostname": "",            // Source DB IP
"database.port": "5444",            // Source DB Port
"database.password": "",            // Source DB user password
"database.dbname": "edb",           // Database name
"bootstrap.ip": "",                 // Broker Server IP
"include.schema.list": "schema1,schema2,schema3",
"schema.history.internal.kafka.bootstrap.servers": "<serverIP>:9092",
"publication.name": "",             // Logical replication publication name
"snapshot.mode": "initial",
"database.history.kafka.bootstrap.servers": "<serverIP>:9092",
"database.history.kafka.topic": "dbhistory.<Topicname>",
"topic.prefix": "",                 // Prefix for Replication topics
"max.batch.size": "2048",
"max.queue.size": 50000,
"poll.interval.ms": "1000",
"producer.batch.size": "32768",
"producer.max.request.size": "10485760"
```

Grant Source Permissions Automatically

Run this to apply necessary grants at source:

Enter comma-separated Table list at `tablelist.config`. These are the tables

which you want to replicate.

```
Example: Schema1.Table1,
         Schema2.Table2,
         Schema2.Table3
```

Then Run `tabletorepl.jar` for Necessary Privileges:

```
java -jar tabletorepl.jar -s <path_to_publication.config> -tables <path_to_ta
blelist.config>
```

For command help Run

```
java -jar tabletorepl.jar -help
```

Step 3: Configure Destination Database

Edit `subscription.config` with the following keys:

```
"name": "",   // Unique identifier
"tasks.max": "1",
"connection.url": "jdbc:postgresql://<Dest_ServerIP>:5444/edb?user=helyx&pass
word=<password>",
"bootstrap.ip": "",  // Kafka IP
"retry.backoff.ms": "1000",
"consumer.max.poll.records": "1000",
"consumer.fetch.max.bytes": "10485760",
"batch.size": "5000",
"max.retries": "10"
```

> **Note**: User name **helyx** will **NOT** be changed

Step 4: Use CLI Tool for Replication Management

Here are common usages of `serverconfig.jar`:

Action	Command Format
Create publication	`java -jar serverconfig.jar -createpub -s publication.config -tables tablelist.config`
Create subscription	`java -jar serverconfig.jar -createsub -s subscription.config -pub publication.config`

Action	Command Format
Add tables to publication	`java -jar serverconfig.jar -addtabletopub -s publication.config -tables tablelist.config`
Trigger ad-hoc snapshot	`java -jar serverconfig.jar -adhocsnapshot -s publication.config -tables table1,table2`
Monitor publication status	`java -jar serverconfig.jar -status -pub -s publication.config`
Monitor subscription status	`java -jar serverconfig.jar -status -sub -s subscription.config`
Remove publication	`java -jar serverconfig.jar -remove -pub -s publication.config`
Remove subscription	`java -jar serverconfig.jar -remove -sub -s subscription.config`
Remove table from replication	`java -jar serverconfig.jar -removetablefromrepl -s publication.config -tables table1,table2`
Encrypt password	`java -jar serverconfig.jar -encryptPassword -s publication.config`
List connectors	`java -jar serverconfig.jar -listconnectors -s publication.config`
List tasks	`java -jar serverconfig.jar -listtask -s publication.config`
Lag statistics	`java -jar serverconfig.jar -lagstat -s subscription.config`
List replicated tables	`java -jar serverconfig.jar -tablelist -s publication.config`

Step 5: Set Up Destination Database (Manual Grants)

On **destination DB**, execute:

```
CREATE ROLE helyx WITH LOGIN PASSWORD '<password>';
ALTER ROLE helyx WITH REPLICATION;
GRANT CREATE ON DATABASE <Database Name> TO helyx;
CREATE SCHEMA IF NOT EXISTS helyx AUTHORIZATION helyx;

GRANT USAGE ON SCHEMA <Custom Schema Name> TO helyx;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA <Custom Schema N
ame> TO helyx;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO helyx;
GRANT CREATE ON SCHEMA <Custom Schema Name> TO helyx;
GRANT CREATE ON SCHEMA public TO helyx;
ALTER SCHEMA <Custom Schema Name> OWNER TO helyx;
ALTER SCHEMA public OWNER TO helyx;
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA <Custom Schema Name> GRANT ALL ON TABLES T
O helyx;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO helyx;
ALTER DEFAULT PRIVILEGES IN SCHEMA <Custom Schema Name> GRANT USAGE ON SEQUEN
CES TO helyx;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT USAGE ON SEQUENCES TO helyx;
ALTER DEFAULT PRIVILEGES IN SCHEMA <Custom Schema Name> GRANT SELECT, INSERT,
UPDATE, DELETE ON TABLES TO helyx;
```

Step 6: Use CLI Tool for Starting Replication between servers:

Action	Description
Create publication	Creates a logical publication on the source DB using `publication.config` and tables from `tablelist.config`.
Create subscription	Sets up the sink connector using `subscription.config`, and links it to the Kafka topics defined in the publication.
Add tables to publication	Adds new tables to the existing source-side publication so that changes in those tables start replicating.
Trigger ad-hoc snapshot	Triggers a one-time snapshot for specified tables, even after initial replication setup. Useful for re-syncing.
Monitor publication status	Shows current status of the publication connector (active, paused, failed, etc.).
Monitor subscription status	Displays the running status of the Subscription and its tasks.
Remove publication	Completely Stops removes the publication and associated configuration.
Remove subscription	Stops and deletes the Subscription configuration.
Remove table from replication	Removes selected tables from the ongoing replication setup.
Encrypt password	Encrypts the password inside `publication.config` for secure usage. Replaces plaintext with secure token.
List connectors	Lists all running Kafka connectors on the connect cluster. Helps in validation and monitoring.

Action	Description
List tasks	Displays tasks of a connector, their IDs, and current status (running, failed, paused).
Lag statistics	Shows replication lag metrics to monitor delay in syncing data to the destination.
List replicated tables	Lists all tables currently under replication from the source config.

📌 Command Reference:

```
java -jar serverconfig.jar -createpub -s publication.config -tables tablelist
.config
java -jar serverconfig.jar -createsub -s subscription.config -pub publication
.config
java -jar serverconfig.jar -addtabletopub -s publication.config -tables table
list.config
java -jar serverconfig.jar -adhocsnapshot -s publication.config -tables table
1,table2
java -jar serverconfig.jar -status -pub -s publication.config
java -jar serverconfig.jar -status -sub -s subscription.config
java -jar serverconfig.jar -remove -pub -s publication.config
java -jar serverconfig.jar -remove -sub -s subscription.config
java -jar serverconfig.jar -removetablefromrepl -s publication.config -tables
table1,table2
java -jar serverconfig.jar -encryptPassword -s publication.config
java -jar serverconfig.jar -listconnectors -s publication.config
java -jar serverconfig.jar -listtask -s publication.config
java -jar serverconfig.jar -lagstat -s subscription.config
java -jar serverconfig.jar -tablelist -s publication.config
```

✅ Once *configuration* and *grants* are ready, start the replication:

▶️ Start the Publication

```
java -jar serverconfig.jar -createpub -s publication.config -tables tablelist
.config
```

▶️ Start the Subscription

```
java -jar serverconfig.jar -createsub -s subscription.config -pub publication
.config
```

▶️ You can monitor the connector statuses:

```
java -jar serverconfig.jar -status -pub -s publication.config
java -jar serverconfig.jar -status -sub -s subscription.config
```

🚀 Final Checklist

Item	Status
Replication Server, Connect, Zookeeper, Schema Registry running	✅
Source `publication.config` configured	✅
Destination `subscription.config` configured	✅
Grants applied to source and destination DBs	✅
Topics appearing in Kafka	✅
Messages flowing to sink	✅

🔄 Schema Evolution: Add Column to Replicated Table

If you want to add a new column to a table that is already being replicated, follow these steps carefully:

🧬 Purpose:

To safely add a new column to an existing replicated table and synchronize the schema with downstream systems.

✅ Steps:

5. **Stop the Connect Worker service**:

   ```
   systemctl stop connect-distributed
   ```

   This ensures that no replication occurs during the schema update.

6. **Add the column in the source database**: Connect to the source database and run:

   ```
   ALTER TABLE <schema>.<table_name> ADD COLUMN <new_column_name> <datatype>;
   ```

7. **Start the Connect Worker service**:

```
systemctl start connect-distributed
```

The connector will reload the schema change and start applying changes again.

8. **Trigger an ad-hoc snapshot for the updated table**: This will send the entire current state of the table (with the new column) to the existing topic.

```
java -jar serverconfig.jar -adhocsnapshot -s publication.config -tables
<table_name>
```

This ensures the new column is picked up and replicated to the destination database.

Optional: Monitoring & Troubleshooting

- Check Kafka logs:

```
tail -f /var/log/kafka/kafka.log
```

- Validate Avro schema at: `http://<schema-registry>:8081/subjects`
- Monitor replication lag:

```
java -jar serverconfig.jar -lagstat -s subscription.config
```


Optional: Monitoring & Troubleshooting

- Check Kafka logs:

```
tail -f /var/log/kafka/kafka.log
```

- Monitor replication lag:

```
java -jar serverconfig.jar -lagstat -s subscription.config
```

Need Help?

Run:

```
java -jar serverconfig.jar -help
```

For all supported actions.

Note: Always test on a staging environment before applying to production.

Happy Replicating! 🚀